Physics-Constrained Machine Learning

Concepts, Tools, and Algorithms

Victor M. Zavala and Angan Mukherjee

Department of Chemical and Biological Engineering
University of Wisconsin-Madison

victor.zavala@wisc.edu

Module 1: Machine Learning Architectures



About the Instructors



Victor M. Zavala

- Baldovin-DaPra Professor, Chemical and Biological Engineering, UW-Madison.
- Joint appointment, Mathematics and Computer Science Division, Argonne National Laboratory.
- Ph.D. Chemical Engineering, Carnegie Mellon University (2008); B.Sc. Chemical Engineering, Universidad Iberoamericana (2003).
- Research: computational mathematics, statistics, optimization, and machine learning for chemical and energy systems.
- Author of *Statistics for Chemical Engineers* (Cambridge University Press, 2025); developer of Plasmo.jl, MadNLP.jl, and InfiniteOpt.jl.



Angan Mukherjee

- Postdoctoral Research Associate, Chemical and Biological Engineering, UW-Madison (since 2024).
- Ph.D. Chemical Engineering, West Virginia University (2024; advisor: Debangsu Bhattacharyya); B.E. Chemical Engineering, Jadavpur University (2019).
- Research: physics-constrained ML, uncertainty-aware modeling, digital twins, and topological data analysis.
- Developed mass-, energy-, and thermodynamics-constrained neural networks and open-source software for physics-constrained ML.

Workshop Outline

Module	Topics	Duration
1. ML Architectures	NNs, neural ODEs, GPs	50 min
2. Physics-Informed ML	Additive, informed, constrained; filtering	45 min
3. Estimation and UQ	MLE, MAP, Laplace, VI, GP propagation	55 min
4. Implementation and Case Studies	Tools, LLMs, applications	30 min
Total		**3 hr**
Hands-on tutorials (optional)	Constrained NNs, neural ODEs, VI	separate

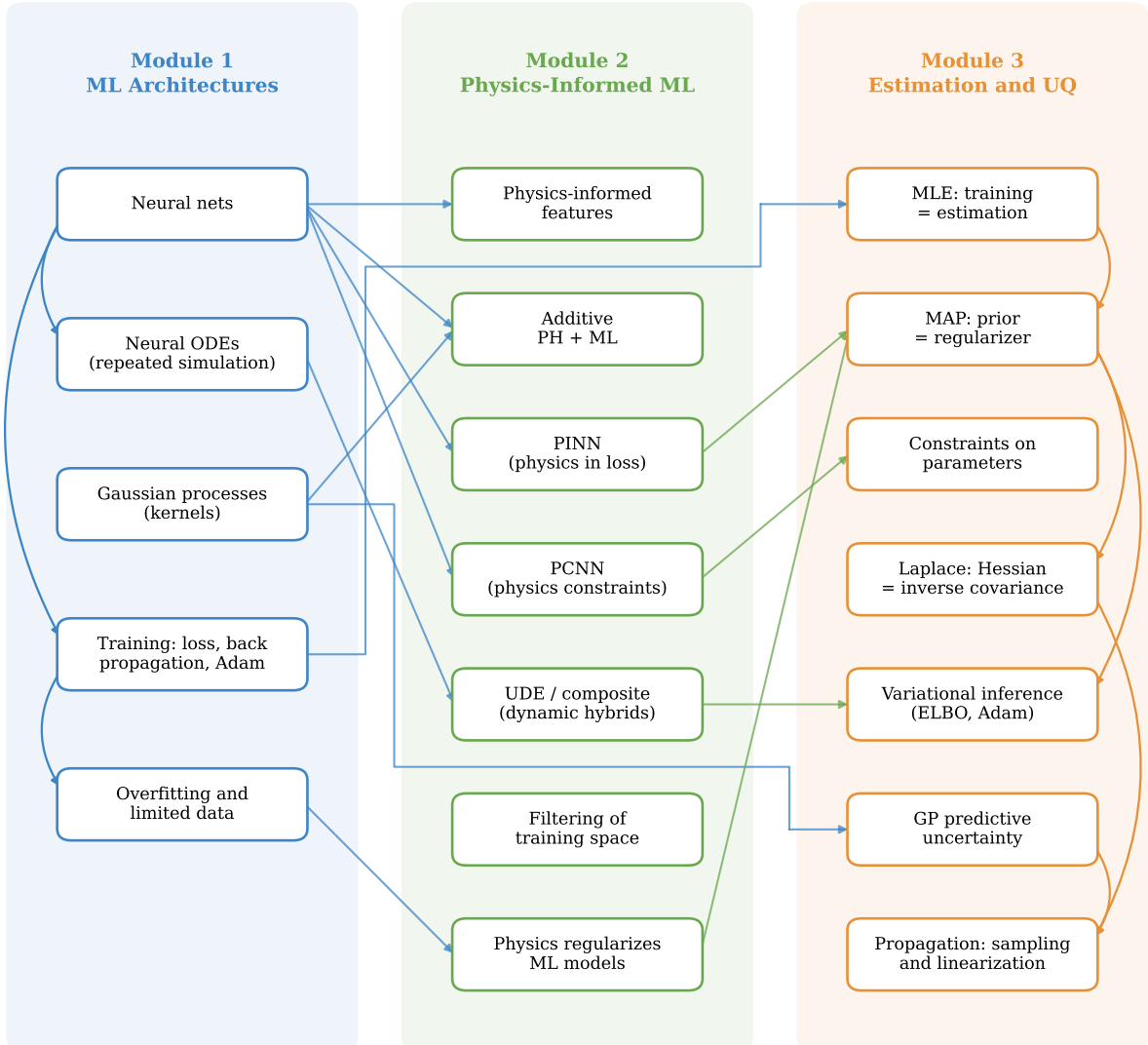
- Each module is integrated with Jupyter notebooks that implement its concepts.
- Running examples: CSTR (Modules 1-2) and membrane separator (Modules 2-3).

Learning Objectives

By the end of the workshop, you will be able to:

- Build NNs, neural ODEs, and GPs: flexible, powerful, and conceptually different tools to model complex systems from data (Module 1).
- Use physics to regularize ML models (prone to overfitting with limited data) and to narrow down the training space (Module 2).
- Estimate models with MLE and Bayesian methods; quantify uncertainty with Laplace, VI, and GPs, and propagate it through functions (Module 3).
- Apply physics-constrained ML to reliable decisions: property prediction, dynamic modeling, Bayesian optimization, and experimental design (Module 4).
- Implement them with open-source tools and LLMs; the key is understanding the concepts.

Knowledge Graph of the Workshop



Machine Learning Models

- Machine learning (ML) is a powerful tool to model systems *directly from data*.
- Given input-output observations (x_ω, y_ω), $\omega \in \mathcal{S}$, we seek a model:

$$y = \mathrm{ML}_\theta(x)$$

  that captures structural relationships between inputs and outputs.
- ML models capture complex multivariable nonlinearities *without* need to postulate a mechanism (input-output model structure).
- This is particularly useful when physical knowledge is incomplete or when physical models are too expensive to evaluate.
- We will see that NNs and GPs are flexible and powerful tools that capture the behavior of complex systems (e.g., a nonlinear reactor).
- Applications in chemical engineering:
  - Property prediction (e.g., molecules, solvents, materials)
  - Dynamic modeling (e.g., reactors, bioreactors, microbial communities)
  - Surrogate models for optimization, control, and experimental design

Parametric and Nonparametric Models

- ML models can be broadly classified as *parametric* or *nonparametric*.
- **Parametric models** (e.g., NNs): model $m(\theta, x)$ has a fixed set of params θ learned from data; predictions only need θ (data can be discarded).
- **Nonparametric models** (e.g., Gaussian processes): model has no fixed params θ; predictions are made directly from data (model complexity grows with data).
- In parametric models we generate alternative models by tuning (varying) params, in nonparametric models we generate alternatives via sampling.
- CSTR example: the NN has 42 params, each GP only 2 (hyper)params.

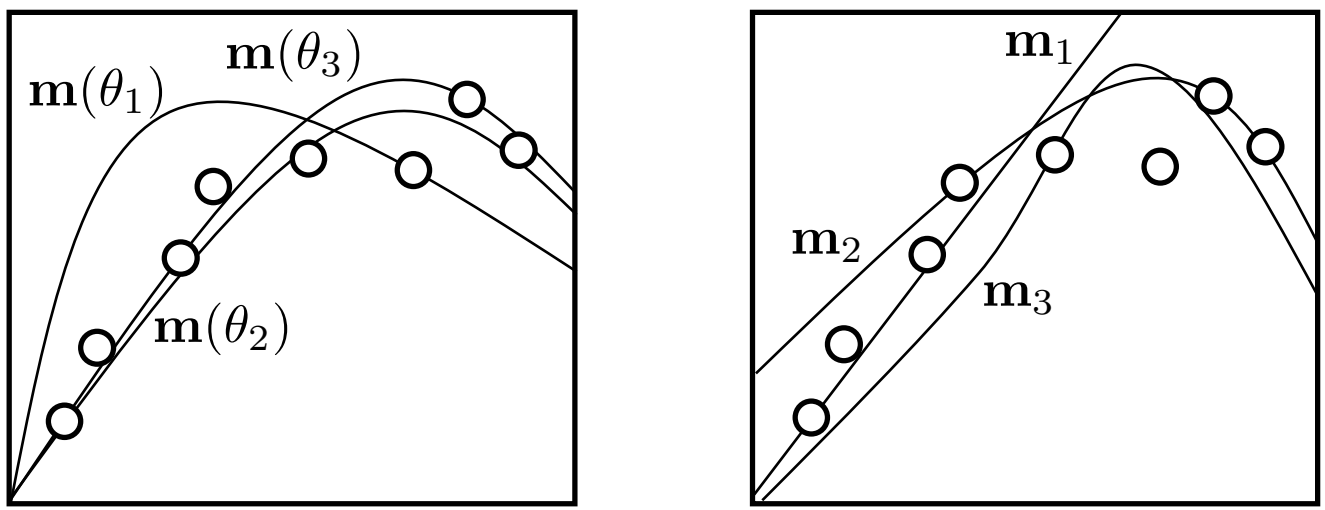
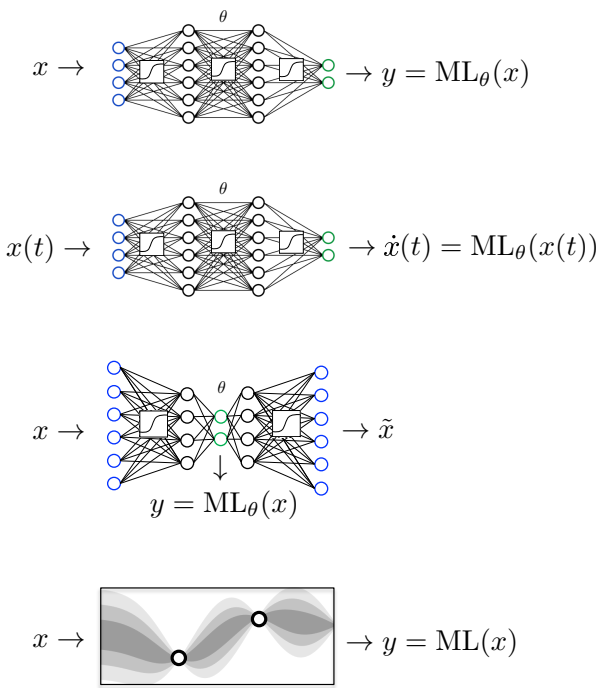


Figure: Alternative models for parametric model $\mathbf{m}(\theta)$ using θ (left). Alternative models for GP using realizations of $\mathbf{m} \sim \mathcal{N}(0, \mathbf{K}(\gamma))$ (right).

Many Types of Neural Networks

- There are many variants of NNs; these are tailored to the type of data available:
  - Feedforward NNs (static data)
  - Recurrent NNs (sequential data, time series)
  - Convolutional NNs (images, fields)
  - Graph NNs (molecules, networks)
  - Autoencoders (high-dimensional data, latent spaces)
  - Neural ODEs (dynamic data, continuous time)
- Focus: feedforward NNs (steady state) and *neural ODEs* (dynamics, can embed physics).



Running Example: CSTR Reactor

- Exothermic reaction $A \to B$ in a cooled CSTR; states are concentration C_A and temperature T, input is coolant temperature T_c:

$$\frac{dC_A}{dt} = \frac{q}{V}(C_{Af} - C_A) - k_0 e^{-E/RT} C_A$$
$$\frac{dT}{dt} = \frac{q}{V}(T_f - T) + \frac{(-\Delta H)}{\rho C_p} k_0 e^{-E/RT} C_A + \frac{UA}{V \rho C_p}(T_c - T)$$

- Reaction rate depends exponentially on temperature; reactor response is highly nonlinear (reactor ignites for T_c above 304 K).
- We use this model *only to generate data*; ML models do not know the model and must learn the behavior from data.
- Steady-state data: 15 experiments with $T_c \in [285, 302]$ K; outputs (C_A, T) measured with noise $\sigma = (0.005\,\mathrm{mol/L},\ 0.5\,\mathrm{K})$.
- Dynamic data: step changes in T_c; (C_A, T) measured every 0.5 min.
- Notebooks: `PCML_Module1_NN_NODE_CSTR` and `PCML_Module1_GP_CSTR`.

Running Example: CSTR Inputs, Outputs, and Parameters

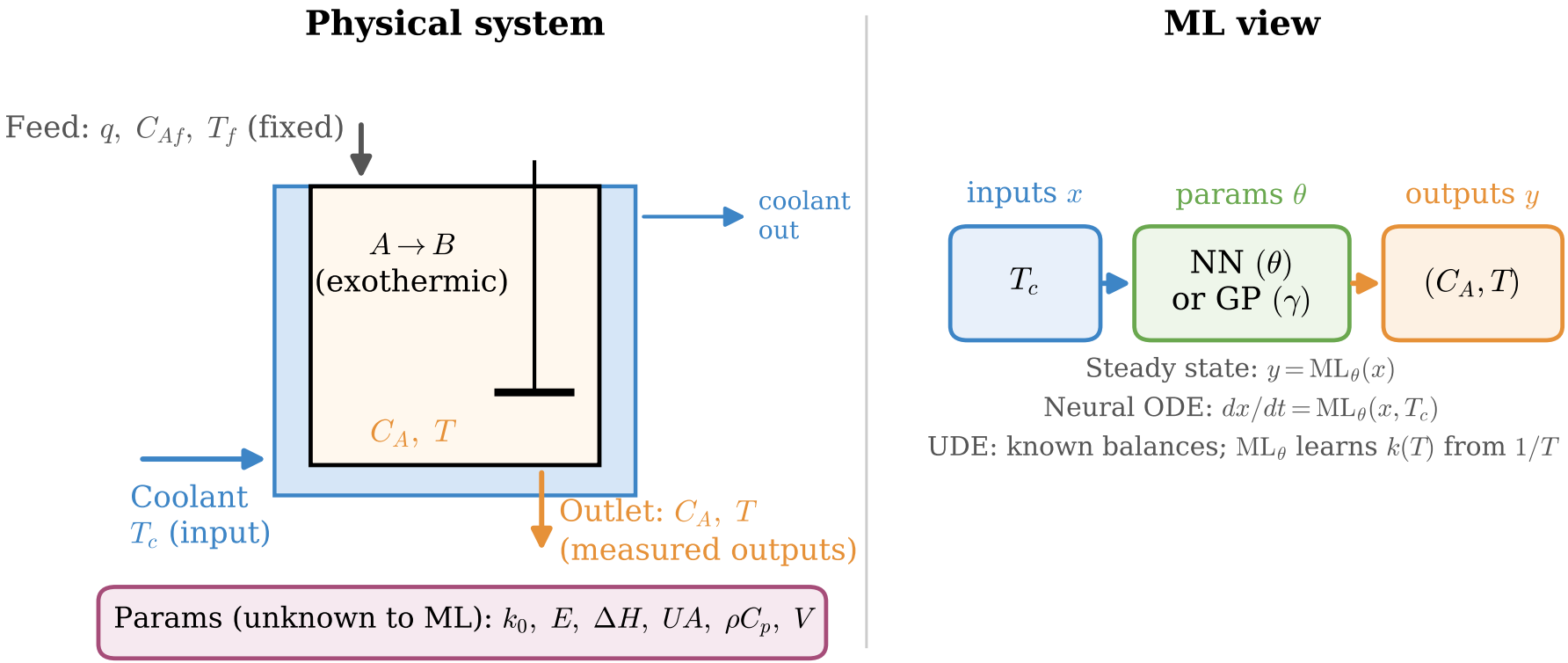


Figure: CSTR as a physical system (left) and as seen by ML models (right).

- Input $x = T_c$; outputs $y = (C_A, T)$; physical params $(k_0, E, \Delta H, UA, \rho C_p, V)$ are unknown to the ML models.
- ML models learn the map from data via own params (θ for NNs, γ for GPs).

Neural Nets

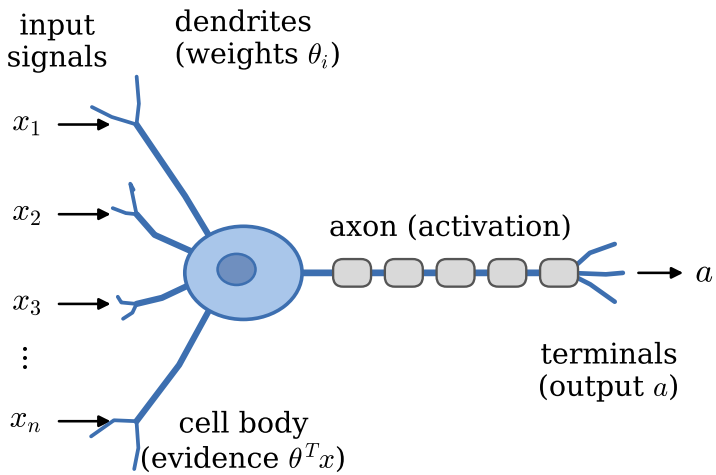
- NNs are motivated by following questions:
  - How do humans learn from observations?
  - How do humans establish connections between variables to make predictions?
- NNs are one of the most general forms of *parametric* models that currently exist.
- NNs mimic how brain (a network of neurons) learns from obs (data from senses).
- NNs are universal; they can capture virtually any relationship between variables.
- On the other hand, flexibility comes at expense of requiring an enormous number of params; one has to verify that NNs generalize & extrapolate well (e.g., predict outside domain of obs).

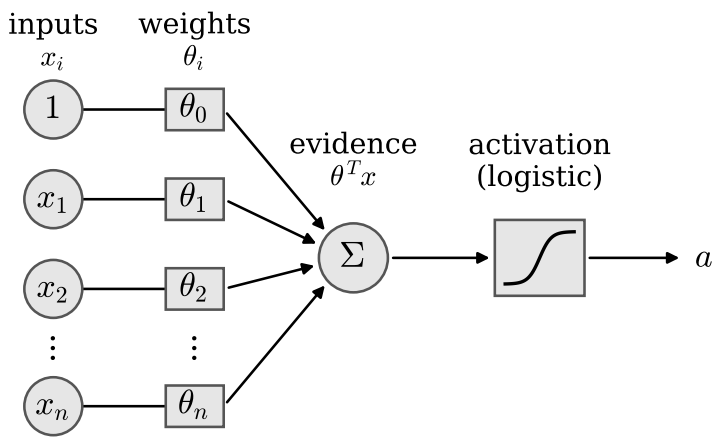
Fully Connected Neural Nets

- Neurons are processing units of brain that interact with one another through signals.
- Math description of a neuron is called a *perceptron*.
- Perceptron generates signal if evidence $\theta^T x = \sum_{i=1}^{n} \theta_i x_i + \theta_0$ is strong enough.
- Here, x_i are inputs with weights θ_i and param θ_0 is *bias*.
- Binary (activate or not) behavior is modeled with an activation function (e.g., logistic):

$$a = \frac{1}{1 + e^{-\theta^T x}}.$$

- There are different types of activation functions (e.g., sigmoid, tanh, ReLU).





Fully Connected Neural Nets

- NN mixes perceptron signals to construct *complex logic*.
- NN architecture consists of input layer, hidden layers, and output layer:
  - Input layer takes input data x_ω, $\omega \in \mathcal{S}$ to generate signals.
  - Hidden layers take signals from previous layer to generate new signals (complex logic).
  - Output layer takes signals from hidden layer to generate final output y_ω, $\omega \in \mathcal{S}$.

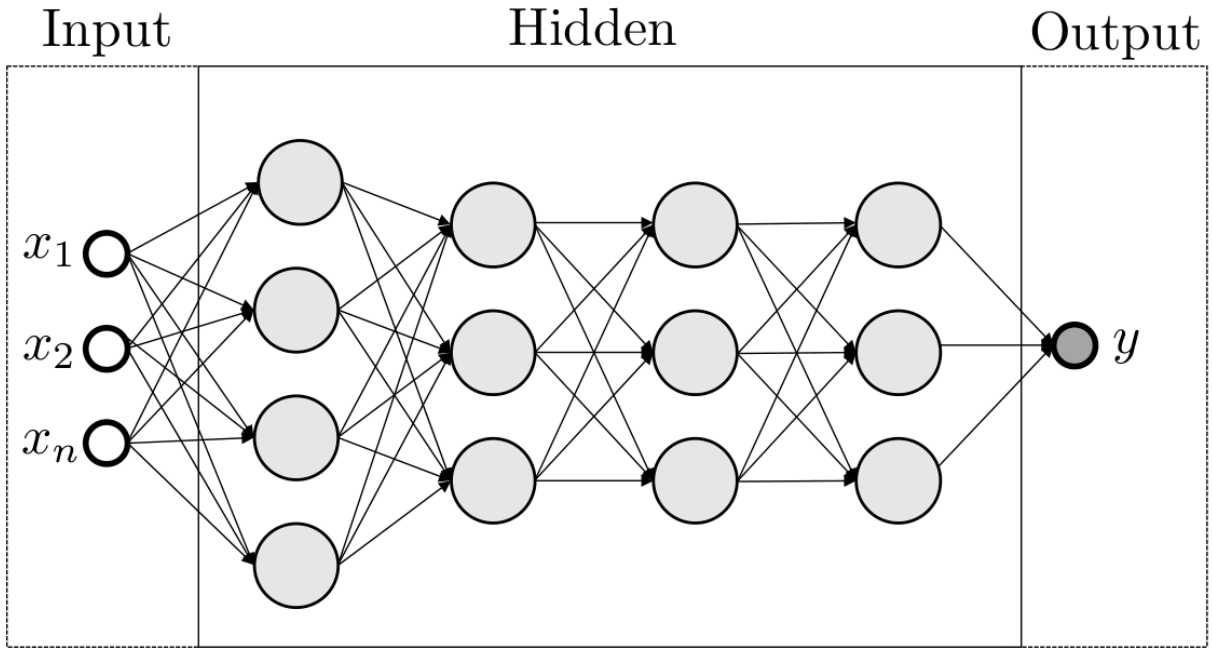


Figure: Sketch of hierarchical structure of a neural network.

Fully Connected Neural Nets

- *Input layer* is composed of n nodes that pass features $x_1, \ldots, x_n$ to the first hidden layer.
- *First hidden layer* has $j = 1, \ldots, n_{H^{(1)}}$ perceptrons with evidence and signal:

$$a_j^{(1)} = \sum_{i=1}^{n} \theta_{j,i}^{H^{(1)}} x_i + \theta_{j,0}^{H^{(1)}}, \quad z_j^{(1)} = h(a_j^{(1)})$$

  where $h(\cdot)$ is the activation function.
- *ℓ-th hidden layer*, $\ell = 2, \ldots, L$, has $k = 1, \ldots, n_H$ perceptrons with evidence and signal:

$$a_k^{(\ell)} = \sum_{j=1}^{n_{H^{(\ell-1)}}} \theta_{k,j}^{H} z_j^{(\ell-1)} + \theta_{k,0}^{H}, \quad z_k^{(\ell)} = h(a_k^{(\ell)})$$

- *Output layer* takes signals of hidden layer as evidence; for regression, we use a linear output:

$$\hat{y} = \sum_{k=1}^{n_{H^{(L)}}} \theta_k^O z_k^{(L)} + \theta_0^O$$

- Propagation of input x through layers gives a nested, *forward propagation* mapping:

$$\hat{y} = \Theta_O h\left(\Theta_{H^{(L)}} \cdots h(\Theta_{H^{(1)}} x)\right) = \mathrm{ML}_\theta(x)$$

  where $\theta = (\Theta_{H^{(1)}}, \ldots, \Theta_{H^{(L)}}, \Theta_O) \in \mathbb{R}^p$ contains all params of the NN.

Training Neural Nets

- Training a NN consists of solving an *optimization problem* (MLE):

$$\min_{\theta} \; L(\theta) = \frac{1}{2} \sum_{\omega \in \mathcal{S}} \left(y_\omega - \mathrm{ML}_\theta(x_\omega)\right)^2 + \kappa \, \rho(\theta)$$

- $L(\theta)$ is the loss and $\kappa \, \rho(\theta)$ a regularization term with weight κ (e.g., $\rho(\theta) = \theta^T \theta$) that mitigates overfitting; it corresponds to a prior (Module 3).

- High-dimensional (thousands to billions of params) and nonconvex (many local minima).

- Solving it requires gradients of the loss with respect to params, $\nabla_\theta L(\theta)$.

- Gradients are computed by *back propagation* (reverse-mode automatic differentiation), marching backward across NN (output to input).

- Cost of computing $\nabla_\theta L(\theta)$ is a small multiple of the cost of evaluating $L(\theta)$; in PyTorch this is done with a single call (`loss.backward()`).

Training Neural Nets: First-Order Methods

- Gradient information is used to progressively refine param estimates using *first-order* methods (gradient descent):

$$\theta_{\ell+1} = \theta_\ell - \eta \, \nabla_\theta L(\theta_\ell)$$

  where η is the step size (learning rate) and ℓ is the iteration counter.

- In practice, gradient is estimated using small subsets of data (mini-batches); this is known as *stochastic* gradient descent.

- Popular variant is Adam, which adapts step for each param using running averages of gradient m_ℓ and of its square v_ℓ:

$$\theta_{\ell+1} = \theta_\ell - \eta \, \frac{\hat{m}_\ell}{\sqrt{\hat{v}_\ell} + \epsilon}$$

- First-order methods are simple and scalable, but convergence is slow (linear) and sensitive to tuning (e.g., learning rate, initialization).

- In our CSTR example, Adam requires thousands of iterations; each iteration is cheap.

Why Not Second-Order Methods?

- Newton's method uses second-order (curvature) information:

$$\theta_{\ell+1} = \theta_\ell - \left[\nabla_\theta^2 L(\theta_\ell)\right]^{-1} \nabla_\theta L(\theta_\ell)$$

- Newton's method converges fast (superlinear), but is challenging to use in NNs:
  - Hessian $\nabla_\theta^2 L(\theta) \in \mathbb{R}^{p \times p}$ is expensive to compute and store (p is large).
  - Factorizing Hessian scales as $\mathcal{O}(p^3)$.
  - Loss is nonconvex; Hessian is indefinite (requires safeguards).
  - Mini-batch gradients are noisy; curvature estimates are unreliable.
- Hessian also carries information about param *uncertainty* (as we will see later); thus, difficulty of computing it makes UQ of NNs challenging.
- Second-order methods (e.g., Ipopt) *can* be used for small NNs and are useful when we need to enforce constraints (as we will see in physics-constrained ML).

Example: NN Model for CSTR PCML_Module1_NN_NODE_CSTR

- NN (one hidden layer, 10 tanh neurons, 42 params) maps T_c to steady state (C_A, T).
- Inputs and outputs are scaled; NN is trained by minimizing loss $L(\theta)$ with Adam (5000 iterations, less than 1 s).
- Inside training range, NN errors are at noise level (RMSE 0.005 mol/L, 0.45 K).
- Outside training range, NN is not reliable; it misses steep temperature rise as reactor approaches ignition (and does not warn us).

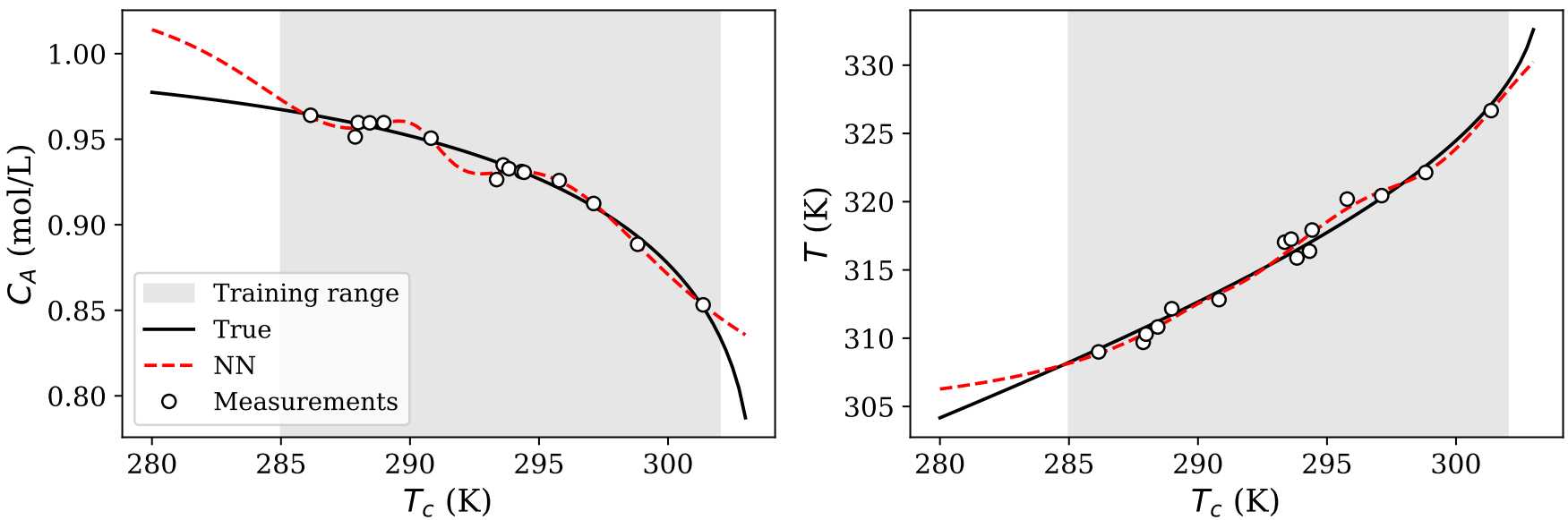


Figure: NN predictions (red) against true steady states (black) and measurements (white dots). Shaded region is training range.

Neural Nets: Strengths and Limitations

- **Strengths:** NNs are flexible and powerful; a few neurons captured the nonlinear CSTR steady state from 15 noisy experiments, without the reactor model.

- **Limitations:**
  - NN params have no physical meaning (difficult to convey prior knowledge).
  - NNs are reliable only where we have data; outside training range predictions can be wrong (CSTR near ignition).
  - NNs can overfit noise; regularization and cross-validation are critical.
  - NNs do not provide uncertainty information directly; uncertainty of params increases with number of params.

- Physics can help NNs generalize; e.g., $y = \mathrm{PH}(x) + \mathrm{ML}_\theta(x)$ (Module 2).

Neural Ordinary Differential Equations

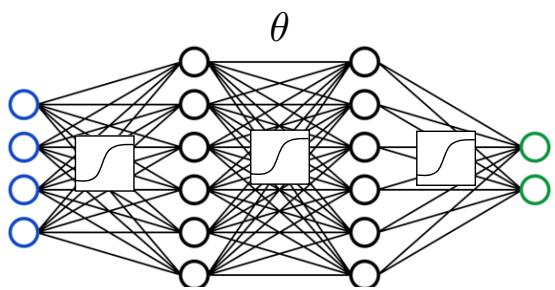
- Many systems are *dynamic* (e.g., reactors, bioreactors, microbial communities).
- A neural ODE (NODE) uses a NN to model the *rate of change* of the states (vector field):

$$\frac{dx(t)}{dt} = \mathrm{ML}_\theta(x(t), u(t)), \quad x(0) = x_0$$

  where $x(t)$ are states and $u(t)$ are inputs.
- Predictions are obtained by integrating the NODE with an ODE solver:

$$\hat{x}(t) = x_0 + \int_0^t \mathrm{ML}_\theta(x(\tau), u(\tau))\, d\tau$$

- NN predicts a derivative; the solver produces a trajectory (continuous-time model).
- CSTR: states $x = (C_A, T)$, input $u = T_c$; NN learns right-hand side of balances.

$$x(t) \rightarrow$$  $$\rightarrow \dot{x}(t) = \mathrm{ML}_\theta(x(t))$$

Training Neural ODEs

- Data consists of measured time trajectories y_ω at times t_ω, $\omega \in \mathcal{S}$.
- For given θ, we simulate the NODE to obtain predicted trajectories $\hat{x}(t;\theta)$.
- NODE is trained by fitting predicted trajectories to data trajectories:

$$\min_\theta \; L(\theta) = \frac{1}{2} \sum_{\omega \in \mathcal{S}} \left(y_\omega - \hat{x}(t_\omega;\theta)\right)^2 + \kappa\, \rho(\theta)$$

- Training thus requires *repeated simulations*: at each iteration we simulate the NODE, compare trajectories with data, and update θ (e.g., using Adam).
- Gradients $\nabla_\theta L(\theta)$ are obtained by back propagation through the simulation.

Why Neural ODEs?

- NODEs are continuous-time models; they handle irregular sampling and predict at any time.
- NODEs connect naturally with physical models, which are often expressed as differential equations (e.g., mass balances, kinetics).
- This makes it possible to embed physics directly in the vector field. For instance, for species abundances $s_i(t)$ in a microbial community:

$$\frac{ds_i(t)}{dt} = s_i(t) \cdot \mathrm{ML}_{\theta,i}(s(t), m(t), u)$$

  ensures that species that are absent remain absent (growth is proportional to abundance).
- NODEs with physics components are also known as universal differential equations (UDEs); we explore these in Modules 2-4.
- Limitations: training requires repeated ODE solves (expensive), stiff dynamics are challenging, and UQ is difficult (many params).

Example: Neural ODE for CSTR PCML_Module1_NN_NODE_CSTR

- Reactor starts at steady state ($T_c = 290$ K); we step T_c to 286, 294, and 300 K and measure (C_A, T) every 0.5 min for 8 min.
- NODE: NN with inputs (C_A, T, T_c), 16 tanh neurons, outputs $(dC_A/dt, dT/dt)$ (98 params).
- NODE is trained by repeated simulation of the three experiments and fitting simulated to measured trajectories (1500 Adam iterations, about 30 s).
- NODE trajectories match training data up to measurement noise.

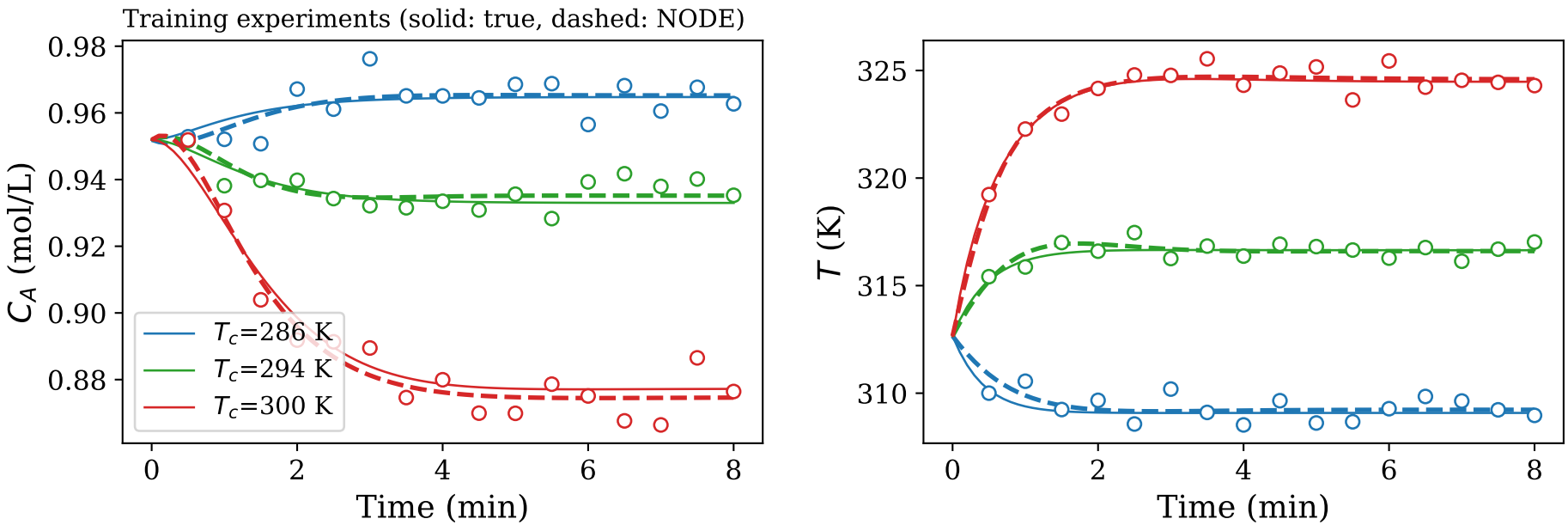


Figure: Training experiments: true trajectories (solid), NODE trajectories (dashed), and measurements (circles).

Example: Neural ODE for CSTR PCML_Module1_NN_NODE_CSTR

- We use NODE to predict response to coolant temperatures not seen in training.
- Interpolation ($T_c = 297$ K): good predictions (RMSE of 0.002 mol/L and 0.36 K).
- Extrapolation ($T_c = 303$ K): reactor is close to ignition and dynamics are qualitatively different; predictions degrade (RMSE of 0.055 mol/L and 3.3 K).
- Like the steady-state NN, NODE is powerful inside data range but blind outside it.

Example: Neural ODE for CSTR PCML_Module1_NN_NODE_CSTR

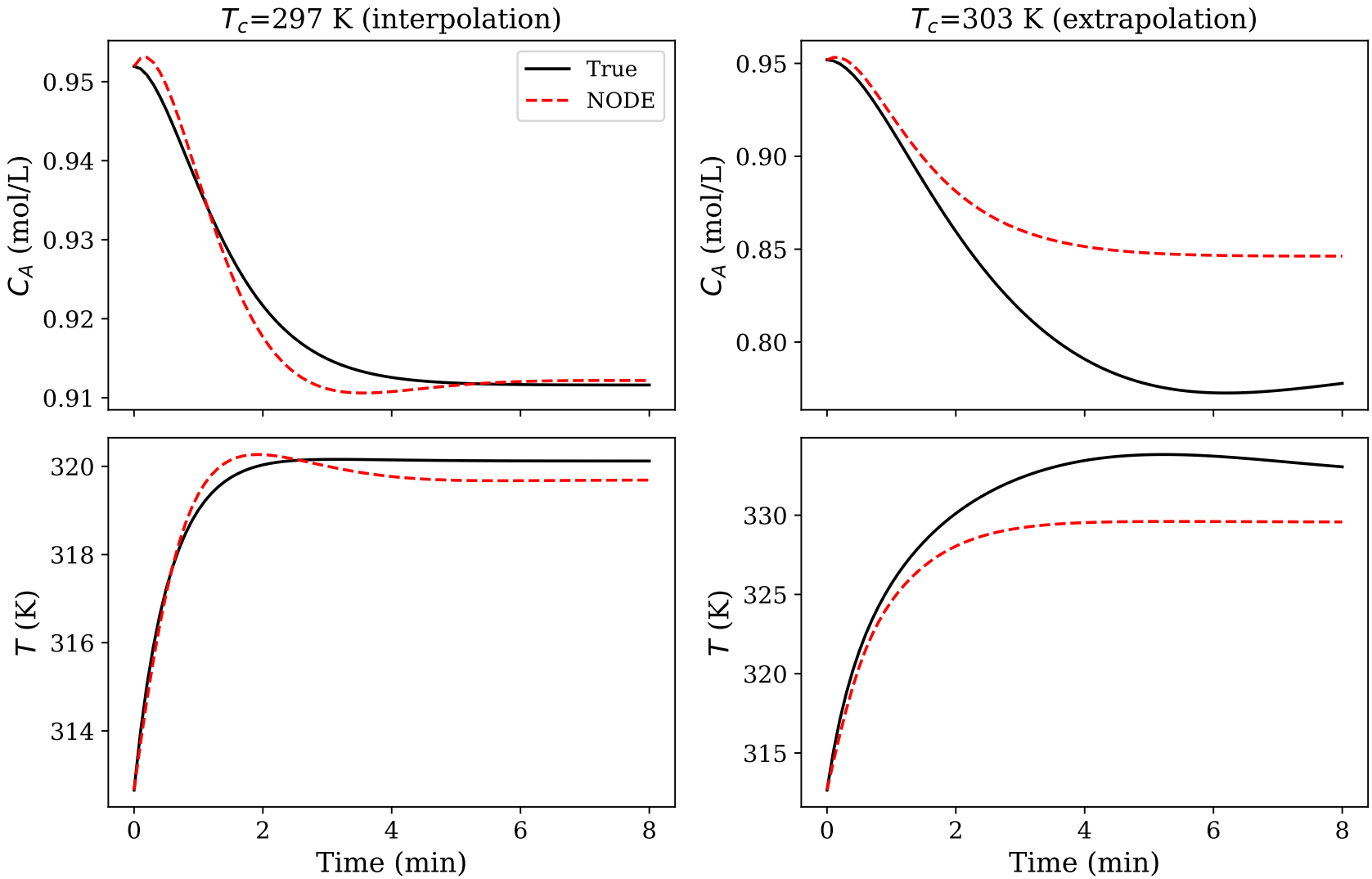


Figure: True (black) and NODE (red) trajectories for interpolation (left) and extrapolation (right).

Gaussian Process Models

- Gaussian process (GP) modeling (kriging) is a *nonparametric* kernel technique.
- In GP modeling, we postulate model of form:

$$y_\omega = m(x_\omega) + \epsilon_\omega,\ \omega \in \mathcal{S}$$

- In vector form:

$$\mathbf{y} = \mathbf{m} + \epsilon$$

- $\mathbf{y} \in \mathbb{R}^S$ contains obs y_ω and $\mathbf{m} \in \mathbb{R}^S$ contains model obs $m(x_\omega)$.
- In GP modeling we treat $\mathbf{m}$ as a random *function* that has no params θ.
- This is key difference over parametric models $\mathbf{m}(\theta, x_\omega)$.

Gaussian Process Models

- Model $\mathbf{m}$ is assumed to be RV $\mathcal{N}(0, \mathbf{K}(\gamma))$.
- Here, $\mathbf{K}(\gamma) = \mathrm{Cov}(\mathbf{m}) \in \mathbb{R}^{S \times S}$ is model covariance and γ are its (hyper)params.
- $\mathbf{K}(\gamma)$ is a matrix function that generates samples of model $\mathbf{m}$.
- Covariance is a kernel matrix with entries:

$$\mathbf{K}_{\omega,\omega'}(\gamma) = k(x_\omega, x_{\omega'}, \gamma),\ \omega, \omega' \in \mathcal{S}$$

  where $k(x_\omega, x_{\omega'}, \gamma)$ is kernel function.

Kernel Functions

- Widely used kernel is radial basis function (RBF):

$$k(x, x', \gamma) = \gamma_0 \exp\left(-\frac{\|x - x'\|^2}{2\gamma_1^2}\right)$$

  where γ_0 is variance and γ_1 is length scale (Matérn kernels are also common).

- Kernel captures similarity between inputs: nearby inputs have correlated outputs.

- Kernel is a *generating function* of basis functions with a handful of params.

- CSTR: one GP per output with RBF kernel; only γ_0 and γ_1 are learned from data.

Training Gaussian Process Models

- One can show that marginal pdf of $\mathbf{y}$ is $\mathcal{N}(0, \mathbf{C}(\gamma))$ with:

$$\mathbf{C}_{\omega,\omega'}(\gamma) = \mathbf{K}_{\omega,\omega'}(\gamma) + \sigma^2 \cdot \delta_{\omega,\omega'}, \quad \omega, \omega' \in \mathcal{S}.$$

  Here, $\delta_{\omega,\omega'} = 1$ if $\omega = \omega'$ and zero otherwise; σ^2 is noise variance.

- GP is trained by estimating $\hat{\gamma}$ using MLE with marginal likelihood $f(\mathbf{y}|\gamma)$:

$$\max_{\gamma} \; -\frac{1}{2}\log|\mathbf{C}(\gamma)| - \frac{1}{2}\mathbf{y}^T\mathbf{C}(\gamma)^{-1}\mathbf{y} - \frac{S}{2}\log 2\pi$$

- Optimization has only a handful of params γ (can use second-order methods, e.g., L-BFGS).
- However, each evaluation requires factorizing $\mathbf{C}(\gamma) \in \mathbb{R}^{S\times S}$, which scales as $\mathcal{O}(S^3)$ (number of data points).
- Problem is nonconvex; multiple restarts are typically used.
- CSTR: 15 data points, so training takes a fraction of a second.

Predictions with Gaussian Process Models

- GP makes predictions using conditional pdf $f(y|\mathbf{y}, \hat{\gamma})$; here, y is predicted output at new point x and $\mathbf{y}$ are training obs.

- One can show that $f(y|\mathbf{y}, \hat{\gamma})$ is Gaussian $\mathcal{N}(\mu(x), \sigma^2(x))$ with:

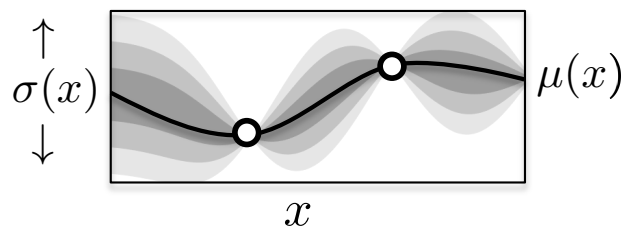
$$\mu(x) = \mathbf{k}(x, \hat{\gamma})^T \mathbf{C}(\hat{\gamma})^{-1} \mathbf{y}$$
$$\sigma^2(x) = c(x, \hat{\gamma}) - \mathbf{k}(x, \hat{\gamma})^T \mathbf{C}(\hat{\gamma})^{-1} \mathbf{k}(x, \hat{\gamma})$$

  where $c(x, \hat{\gamma}) = k(x, x, \hat{\gamma})$ and $\mathbf{k}_\omega(x, \hat{\gamma}) = k(x_\omega, x, \hat{\gamma}),\ \omega \in \mathcal{S}$.

- Predictions are computed directly from data $\mathbf{y}$ (nonparametric).

- GP provides mean prediction $\mu(x)$ and uncertainty $\sigma(x)$; uncertainty is small near data and large away from data.

- CSTR notebook computes $\mu(x)$ and $\sigma^2(x)$ by hand with these formulas and recovers scikit-learn predictions.



Example: GP Model for CSTR PCML_Module1_GP_CSTR

- GP with RBF kernel predicts steady state (C_A, T) from T_c (same data as NN); one GP per output with only 2 hyperparams, trained by MLE.
- Inside training range, GP captures nonlinear map (RMSE of 0.003 mol/L and 0.49 K) and confidence intervals are narrow.
- Outside training range, bands widen: the GP *knows when it does not know*.
- GP misses temperature rise near ignition, but its uncertainty flags unreliable predictions.

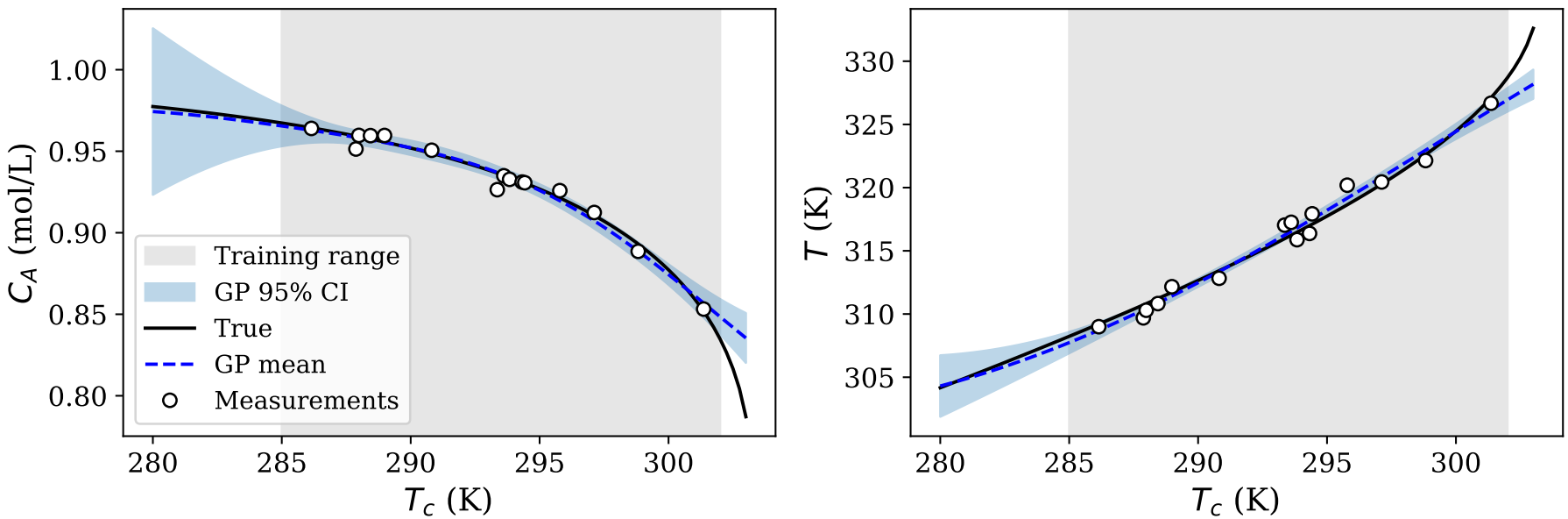


Figure: GP mean (dashed) and 95% confidence interval (blue) against true steady states (black) and measurements (white dots).

Neural Nets vs. Gaussian Processes

	Neural Nets	**Gaussian Processes**
Model type	Parametric $\mathrm{ML}_\theta(x)$	Nonparametric $\mathcal{N}(\mu(x), \sigma^2(x))$
Params	Many weights θ	Handful of hyperparams γ
Training	First-order methods (Adam)	MLE (L-BFGS); $\mathcal{O}(S^3)$
Data	Large datasets, high-dim inputs	Small datasets, low-dim inputs
Predictions	Use θ only	Use all training data
UQ	Complex (Laplace, VI, ensembles)	Easy (built-in $\sigma(x)$)
Dynamics	Natural (neural ODEs)	Less natural
CSTR (inside data)	RMSE 0.005 mol/L, 0.45 K	RMSE 0.003 mol/L, 0.49 K
CSTR (outside data)	No warning	Uncertainty grows

- **Key:** NNs learn a function (optimize params); GPs learn a distribution over functions (condition on data); thus, UQ is easy for GPs, complex for NNs.
- Both can be combined with physics; this is topic of Module 2.

Key Takeaways

- NNs and GPs are *flexible and powerful* modeling tools that capture the behavior of complex systems directly from data.

- In the CSTR, a small NN and a GP learned a highly nonlinear steady-state map from 15 experiments, and a neural ODE learned the reactor dynamics from 3 experiments, without knowing the reactor model.

- The two methodologies are *conceptually very different*:

  - NNs are parametric: we postulate a function $\mathrm{ML}_\theta(x)$ with many params and find a single best θ by optimization (Adam); neural ODEs do this by repeated simulation and fitting of trajectories.
  - GPs are nonparametric: a distribution over functions conditioned on data; a handful of hyperparams are learned and $\sigma(x)$ comes for free.

- Both are reliable inside the range of data but cannot know what they have not seen; physics (Module 2) and uncertainty quantification (Module 3) help address this.

References

- Zavala, V. M. (2025). Statistics for Chemical Engineers. Cambridge University Press. doi:10.1017/9781009541916
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533-536. doi:10.1038/323533a0
- Kingma, D. P., and Ba, J. (2015). Adam: A method for stochastic optimization. ICLR. doi:10.48550/arXiv.1412.6980
- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. (2018). Neural ordinary differential equations. NeurIPS. doi:10.48550/arXiv.1806.07366
- Rasmussen, C. E., and Williams, C. K. I. (2006). Gaussian Processes for Machine Learning. MIT Press. doi:10.7551/mitpress/3206.001.0001
- Thompson, J., Connors, B. M., Zavala, V. M., and Venturelli, O. S. (2026). Physics-constrained neural ordinary differential equation models to discover and predict microbial community dynamics. PNAS, 123(13), e2517661123. doi:10.1073/pnas.2517661123
- Mukherjee, A., and Zavala, V. M. (2026). Physics-constrained machine learning for chemical engineering. Curr. Opin. Chem. Eng., 51, 101228. doi:10.1016/j.coche.2026.101228